

Variationally Correct Operator Learning for Nonlinear Parametric PDEs: Reduced Basis Neural Operators with Local a Posteriori Error Certification

Yuan Qiu Wolfgang Dahmen Peng Chen

yuan.qiu@gatech.edu • dahmen@math.sc.edu • pchen402@gatech.edu



companion paper

Motivation

- **Many-query tasks** (such as uncertainty quantification, inverse problems, etc) for parametric PDEs require repeated evaluations of a parameter-to-solution map.
- **Neural operators** are fast to evaluate surrogate models once trained, but still need many high-fidelity solution snapshots for training if using a data-driven approach.
- **Residual minimization** evaluates the governing equation at predicted solutions, and thus does not need reference solutions for training, which greatly save the offline solves.
- But the way to interpret the governing equation and formulate the residual is quite important. PINNs and PINO's directly sampling the pointwise residual of the original PDE may be not enough error informative. More precisely, a small residual does *not* guarantee a small prediction error in L^2 norm.
- We want to take a closer look at the residual-error relationship and build a residual loss that is tighter guess of the underlying prediction error in a more suitable norm beyond L^2 .

A variationally correct loss

- Meaning: the residual loss value is always uniformly proportional to the squared solution error in the norm determined by the stable variational formulation.
- Denote the parameter by $\mathbf{p}$ and the unknown solution by s which has component of stress tensor σ and displacement u . Suppose the boundary conditions are either Dirichlet or Neumann and boundary data is parameter-independent. We can solve auxiliary problems to lift the boundary data and learn for the corresponding homogeneous problem. This removes the boundary penalty terms in the loss. Then we have decomposition $\sigma = \sigma^\circ + z$, $u = u^\circ + w$.
- Let us denote $s = [\sigma^\circ, u^\circ] \in \mathbb{H} = H(\text{div}) \times H^1$ the solution we aims to predict given the corresponding parameter $\mathbf{p}$. We first reformulate the original PDE into a first-order system (FOS) by introducing the stress tensor as an additional unknown. Then we can define the FOSLS loss as squared L^2 norm of the residual $\mathcal{L}(s; \mathbf{p}) = \|\mathcal{R}(s; \mathbf{p})\|_{L^2}^2$.
- **For linear PDEs** [1, 2] we have constants c_0, C_0 that do not depend on $\mathbf{p}$,

$$c_0 \|s - s^*(\mathbf{p})\|_{\mathbb{H}}^2 \leq \mathcal{L}(s; \mathbf{p}) \leq C_0 \|s - s^*(\mathbf{p})\|_{\mathbb{H}}^2,$$

so the loss value is reliable indicator of the approximation error.

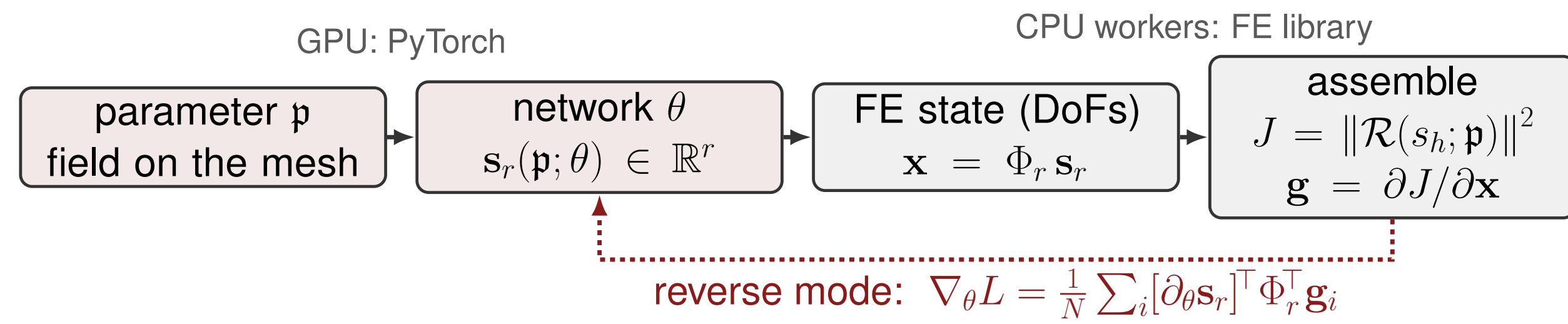
- **For nonlinear PDEs** [3] the same holds when the approximation is close to the solution: fix a stable solution branch and stay within a small neighbourhood of it. If the linearization of the residual is stable there and varies smoothly, then

$$C_{\text{rel}} \|s - s^*(\mathbf{p})\|_{\mathbb{H}}^2 \leq \mathcal{L}(s; \mathbf{p}) \leq C_{\text{eff}} \|s - s^*(\mathbf{p})\|_{\mathbb{H}}^2.$$

The constants follow from that stability, smoothness and the size of the neighbourhood.

Reduced basis neural operator (RBNO)

- **Offline (once):** solve the FOSLS problem for N_{POD} parameters and construct a reduced basis Φ_r of dimension $r < N_{\text{POD}}$.
- **Online (per parameter):** the network returns r coefficients, and the prediction $\Phi_r s_r(\mathbf{p}; \theta)$ is a conforming FE state in the *same* space as the high-fidelity solution.
- **Residual minimization:** Only the N_{POD} offline solves are expensive; the N training parameter residuals are cheap to evaluate; we assume $N_{\text{POD}} \ll N$.



We aim to find the network parameters θ that minimize the empirical loss

$$\theta^* \in \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(\Phi_r s_r(\mathbf{p}_i; \theta); \mathbf{p}_i)$$

The state gradient is an assembled linear form (no FE solve)

$$\mathbf{g}_j = \frac{\partial J}{\partial \mathbf{x}_j} = 2 \left(\mathcal{R}(s_h; \mathbf{p}); A_{s_h, \mathbf{p}} \phi_j \right)_{L^2}$$

where $A_{s_h, \mathbf{p}}$ is the Fréchet derivative of $\mathcal{R}(\cdot; \mathbf{p})$ evaluated at s_h .

- **GPU (PyTorch):** network, Φ_r , chain rule, optimizer (float32).
- **CPU workers (FE library):** cached forms, BCs, J and $\mathbf{g}$ (float64).

Example: compressible Neo-Hookean hyperelasticity

We consider a compressible hyperelasticity problem on the rectangle $\Omega = (0, 2) \times (0, 1)$, which models an elastic body clamped along its left edge and pulled by a traction on its right edge, undergoing deformations large enough that the small-strain assumption no longer applies. More precisely, we consider the following boundary-value problem for the displacement field $u : \Omega \rightarrow \mathbb{R}^2$ and the first Piola–Kirchhoff stress tensor $\sigma : \Omega \rightarrow \mathbb{R}^{2 \times 2}$

$$\begin{cases} \text{div } \sigma + f = 0, & \text{in } \Omega, \\ \sigma = P(I + \nabla u; \mathbf{p}), & \text{in } \Omega, \\ u = u_D, & \text{on } \Gamma_D, \\ \sigma \cdot n = t, & \text{on } \Gamma_N. \end{cases} \quad (1)$$

where $P(F; \mathbf{p}) = \partial_F \psi(F; \mathbf{p})$ (we let $F = I + \nabla u$ to denote the deformation gradient) is the first Piola–Kirchhoff stress tensor, $\psi(F; \mathbf{p})$ is the stored energy density, f is the body force, u_0 is the clamped displacement, t is the traction, and n is the outward unit normal vector.

We assume the material is compressible neo-Hookean, with stored energy density

$$\psi(F) = \frac{\mu}{2} (\text{tr}(F^T F) - 2) - \mu \ln(\det F) + \frac{\lambda}{2} (\ln(\det F))^2,$$

where the Lamé parameters are obtained from the Young's modulus and Poisson ratio $\nu = 0.4$

$$\lambda = \mathbf{p} \nu / ((1 + \nu)(1 - 2\nu)) \quad \text{and} \quad \mu = \mathbf{p} / (2(1 + \nu)).$$

We assume the Young's modulus takes the form of $\mathbf{p} = \mathbf{p}(x) = 1 + \exp(m(x))$, where m is sampled a Gaussian random field $\sim \mathcal{N}(\bar{m}, \mathcal{C})$ with mean $\bar{m} = 0$ and covariance operator $\mathcal{C} := (\delta I - \gamma \Delta)^{-\alpha}$ with $\delta = 1.5$, $\gamma = 0.15$, and $\alpha = 2$. The boundary data are prescribed as

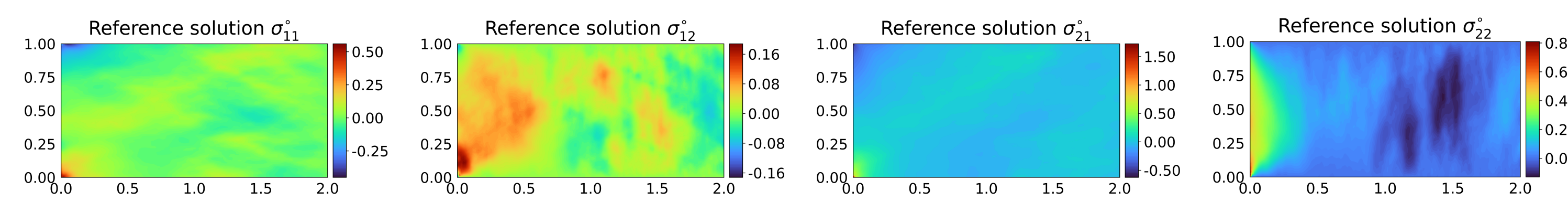
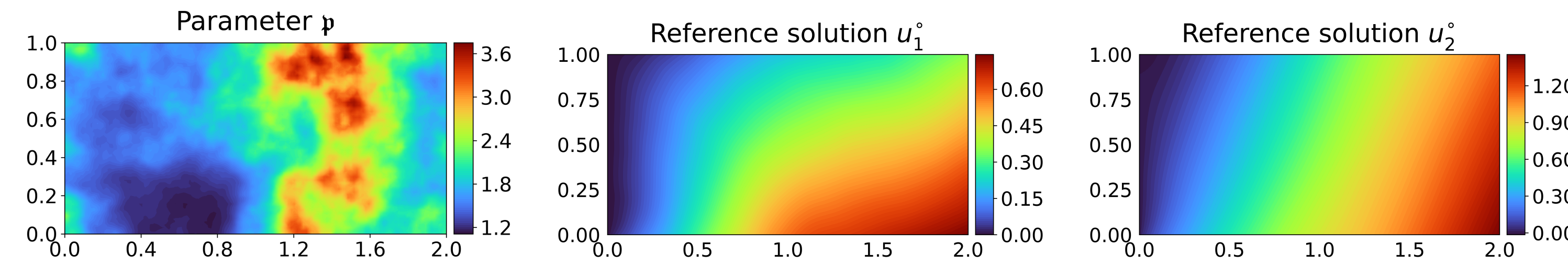
$$u_0(x) = (0, 0)^T, \quad x \in \Gamma_{\text{left}},$$

$$t(x) = \begin{cases} (0, 0)^T, & x \in \Gamma_{\text{top}} \cup \Gamma_{\text{bottom}}, \\ (0.6 \exp(-(x_2 - 0.5)^2/4), 0.3(1 + x_2/10))^T, & x \in \Gamma_{\text{right}}, \end{cases}$$

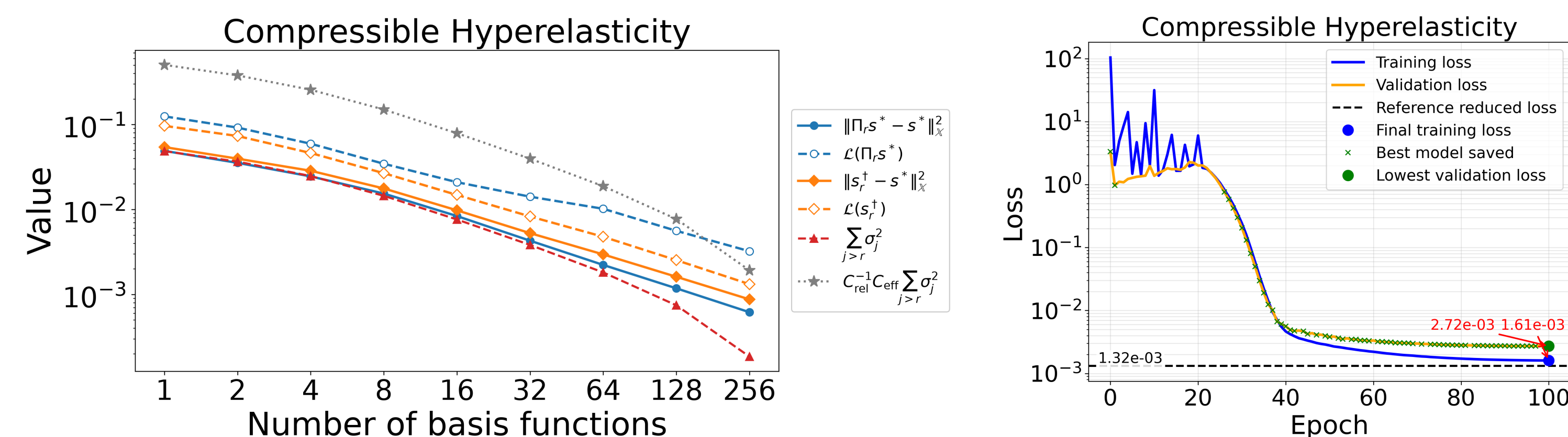
Lifting the boundary data by $u = u^\circ + w$, $\sigma = \sigma^\circ + z$, the FOSLS residual is given by

$$\mathcal{R}([\sigma^\circ, u^\circ]; \mathbf{p}) = \begin{pmatrix} \sigma^\circ + z - P(I + \nabla(u^\circ + w); \mathbf{p}) \\ \text{div } \sigma^\circ + f \end{pmatrix} \text{ on } \mathbb{H} = H_{0, \Gamma_N}(\text{div})^{2 \times 2} \times H_{0, \Gamma_D}^1(\Omega)^2.$$

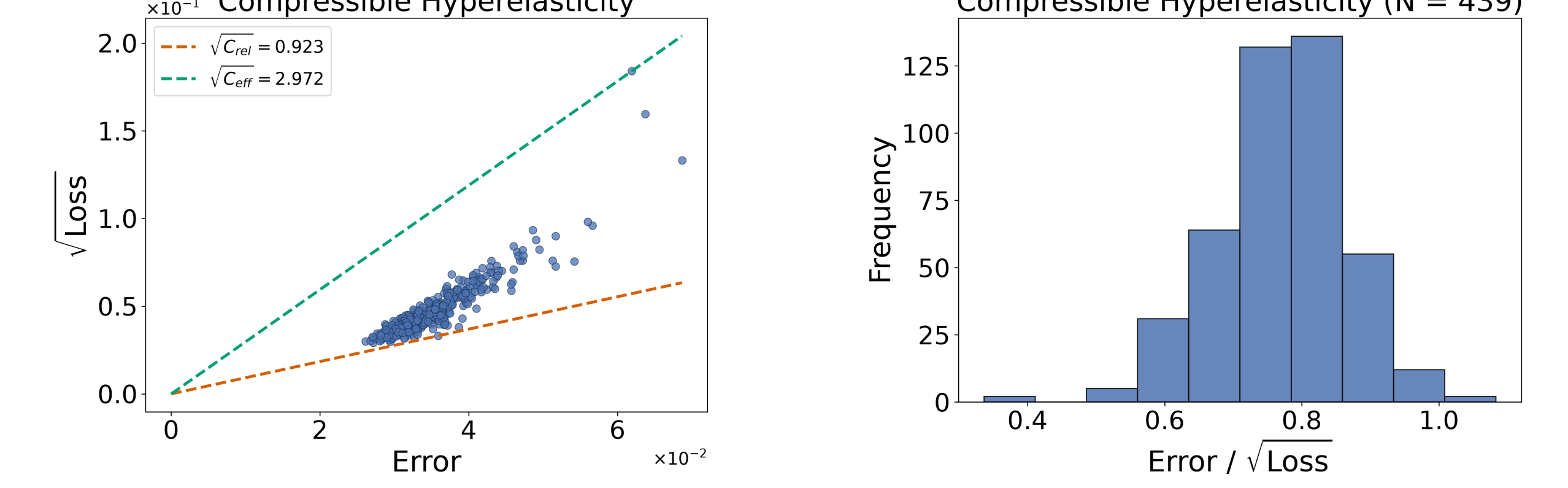
- FE discretization: 128×64 mesh, choose RT element of order 1 for σ and CG element of order 2 for u , resulting in 229,376 DoFs.
- FE solve (using FEniCSx [4]): load-stepping continuation method to generate the snapshots.
- RBNO: use 1000 snapshots to build $r = 256$ POD modes; use CNN encoder, trained with 5000 parameter samples for 100 epochs in 1.5 h using one A1000 GPU and 50 CPU workers.



Visualization of parameter-to-solution map $\mathbf{p}_h \mapsto [u_h^\circ(\mathbf{p}_h), \sigma_h^\circ(\mathbf{p}_h)]$ at a random parameter sample $\mathbf{p}_h$ for the compressible hyperelasticity equation.



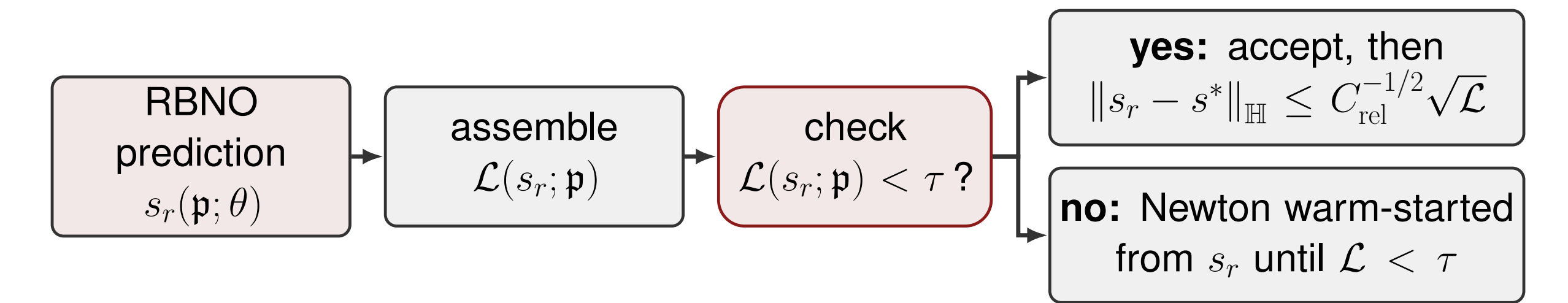
The loss certifies the error



$\sqrt{\mathcal{L}}$ versus the $\mathbb{H}$ -error lies in the cone $\sqrt{C_{\text{rel}}} = 0.923 \leq \sqrt{\mathcal{L}}/\text{err} \leq \sqrt{C_{\text{eff}}} = 2.972$ (left); $\text{err}/\sqrt{\mathcal{L}}$ concentrates in $[0.6, 0.9]$ (right): a tight, computable error indicator for the prediction.

Refine the prediction using solver

If downstream tasks require a more accurate solution, we can use the RBNO prediction as a warm start for the high-fidelity solver. The FOSLS loss is used to gate whether to accept the prediction or refine it with Newton's method.



Accuracy and computational cost comparison

Method (mean over 439 test samples)	rel. err. $L^2 \times L^2$	rel. err. $H(\text{div}) \times H^1$	residual loss
PCA-Net[5]	3.6e-2	2.8e-1	4.3e-1
FNO[6]	6.3e-3	4.2e-1	3.9
FC-PINO[7]	1.4e-2	4.2e-1	6.5e-3
RBNO	2.7e-2	5.0e-2	2.4e-3

Per parameter sample	wall clock time
RBNO inference	0.004 s
FOSLS residual + state-gradient assembly	0.083 s
Newton solve warm-started from the RBNO prediction	12.5 s
Newton solve warm-started from the mean-parameter solution	217 s

Baselines with small L^2 error carry $5\text{--}8 \times$ larger error in the model-compliant $H(\text{div}) \times H^1$ norm; warm-start from the RBNO prediction makes the high-fidelity solve $17 \times$ faster.

Take-aways

- **Variational correct loss:** the FOSLS loss on conforming first-order states is a posteriori error certificate and the gate for Newton refinement.
- **GPU-CPU coupling for residual minimization:** the network is trained on GPU, while the residual and its gradient are assembled on CPU workers using FEniCSx library.
- **Possible next steps:** hyper-reduction, localized or nonlinear reduced spaces, multi-scaled representations, space-time FOSLS, an MFEM realization with GPU assembly.

References and acknowledgments

- [1] Yuan Qiu, Wolfgang Dahmen, and Peng Chen. Variationally correct operator learning: Reduced basis neural operator with a posteriori error estimation. *Computer Methods in Applied Mechanics and Engineering*, 2026. In press, arXiv:2512.21319.
- [2] Markus Bachmayr, Wolfgang Dahmen, and Mathias Oster. Variationally correct neural residual regression for parametric pdes. *IMA Journal of Numerical Analysis*, 2025.
- [3] Yuan Qiu, Wolfgang Dahmen, and Peng Chen. Variationally correct operator learning for nonlinear parametric pdes: Reduced basis neural operators with local a posteriori error certification, 2026. In preparation.
- [4] Igor A Baratta, Joseph P Dean, Jørgen S Dokken, Michal Habera, Jack Hale, Chris N Richardson, Marie E Rognes, Matthew W Scroggs, Nathan Sime, and Garth N Wells. Dolfinx: the next generation fenics problem solving environment. 2023.
- [5] Kaushik Bhattacharya, Bamdad Hosseini, Nikola B Kovachki, and Andrew M Stuart. Model reduction and neural networks for parametric pdes. *The SMAI journal of computational mathematics*, 7:121–157, 2021.
- [6] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020.
- [7] Adarsh Ganeshram, Haydn Maust, Valentin Duruisseaux, Zongyi Li, Yixuan Wang, Daniel Leibovici, Oscar Bruno, Thomas Hou, and Anima Anandkumar. FC-PINO: High precision physics-informed neural operators via fourier continuation. *arXiv preprint arXiv:2211.15960*, 2022.

Supported in part by NSF grants DMS-2245111, DMS-2245097, DMS-2038080 and by the German Research Foundation, CRC 1481.